

OpenADMET PXR Methodology Report

Environment

Software

Python in dedicated Conda environments under Windows. Code generated using Google Gemini.

Hardware

Very limited. CPU only, Up to 12Gb available memory (optimistically) out of 32Gb installed. Attempts to use the laptop GPU met with limited success and frequent crashes.

Method

The training script implements a **Chemprop v2 hybrid architecture** optimized for regression using multi-modal feature fusion and a strict validation pipeline.

1. Data Prep & Applicability Domain

- **RDKit Filter:** Restricts the dataset to an explicit molecular weight range of 160 - 600.
- **Target Normalization:** Applies standard z-score normalization to the pEC50 values per fold. The calculated statistics are bound to an integrated `UnscaleTransform` layer inside the predictor, ensuring model outputs are automatically inverted back to the true scale during evaluation and inference.

2. Multi-Modal Feature Fusion

- **Message Passing:** Uses standard `BondMessagePassing` combined with `MeanAggregation` to extract 2D graph representations.
- **Extra Atom/Bond Features (`X_d`):** Augments the graph representation by concatenating a 2048-dimensional pre-trained continuous embedding (`CheMeleonFingerprint`) directly to the pooled graph vector before it enters the regression head.
- **Caching:** Computes an MD5 hash of the SMILES strings to cache the heavy 2048-dim features locally as a `.npz` file, bypassing redundant descriptor generation on restart.

3. Training & Validation Setup

- **Regression Head:** Passes the combined graph and CheMeleon features into a standard `RegressionFFN`, with a dynamic loss criterion set to MAE.
- **Cross-Validation:** Wraps execution in a strict **10-fold cross-validation loop** (`KFold`, `shuffle=True`).
- **Pacing:** Handled via PyTorch Lightning with `EarlyStopping` monitoring `val_loss` (patience = 10 epochs). Tensor Core utilization is optimized for your target hardware via `torch.set_float32_matmul_precision('medium')`.

Things that didn't work well

- The training data error was proportional to the EC50 in linear space. The loss function was tuned to minimize the error in linear space, not in log space. This resulted in higher RAE, and was discarded.
- Minimizing MAE outperformed MSE.
- Stacking was not attempted